

Instructions for the Leaf Classification Problem

TKO 3120 Machine Learning and Pattern Recognition 2026

Welgamage Supun
Doctoral Researcher
Department of Computing
svwelg@utu.fi



**UNIVERSITY
OF TURKU**

Exercise task in general

- The task is to classify leaf images using supervised machine learning
- 3 different leaf classes of images, one label per image
- Gather a training set by extracting several features from the images
- Train classifiers to identify the classes of the images

Ridge Classifier

Random Forest

Multi-Layer Perceptron

- Find the best hyperparameters for the classifiers
- Estimate the performance of each classifier
- Coding with Python
- Individual exercise to be submitted by **15 March**
- Please note: Project work must be **submitted before** taking the e-exam.
- A Jupyter Notebook template, image folders for leafs, and instructions will be found on the Moodle course page



**UNIVERSITY
OF TURKU**

Anaconda

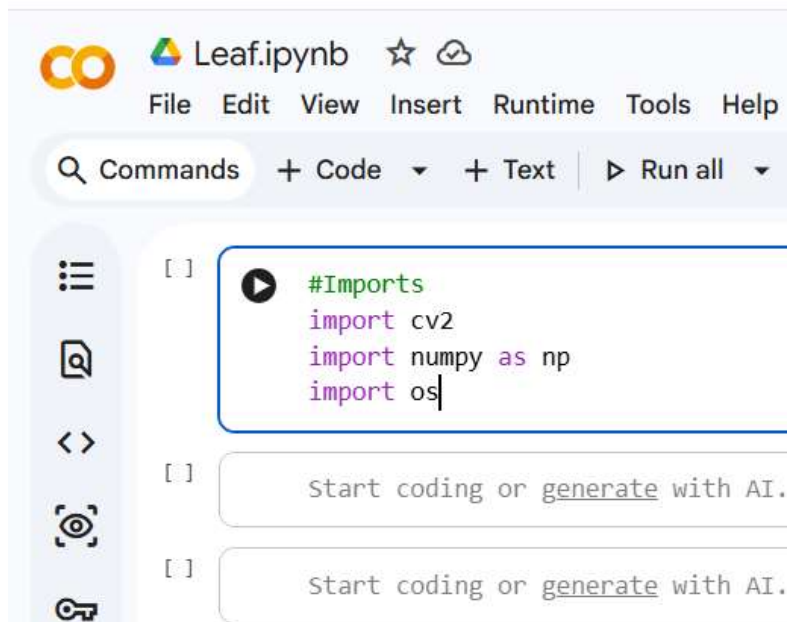
- Install Miniconda or Anaconda Distribution to your computer (Python 3)
 - <https://conda.io/projects/conda/en/latest/user-guide/install/index.html>
 - Includes NumPy, Pandas, Jupyter, scikit-learn, scikit-image, Matplotlib
 - You may also use other modules if needed
- Tutorials
 - Jupyter Notebook
 - <https://www.dataquest.io/blog/jupyter-notebook-tutorial/>
 - Python 3
 - <https://www.tutorialspoint.com/python3/>
 - Scikit-learn
 - https://scikit-learn.org/stable/user_guide.html
 - Scikit-image
 - <https://scikit-image.org/docs/dev/index.html>



**UNIVERSITY
OF TURKU**

Google Colaboratory

- Write and execute Python in your browser
- No configuration required
- Free access to GPUs
- Easy sharing
 - <https://colab.research.google.com/>



**UNIVERSITY
OF TURKU**

Dataset

- Three leaf classes: Basil, Lemon, Chinar
- RGB images
- Each image processed independently
- Data source is Kaggle



**UNIVERSITY
OF TURKU**

Data preparation

- Read the images from the leaf
- RGB images
 - Stored as 3 dimensional arrays
 - [height, width, color channel]
 - Pixel values as 8-bit unsigned integers: [0,255]
- Change the images into grayscale (2-dimensional array) and reduce the quantization level, e.g. to 8 levels, i.e. pixel values [0,7]
- Calculate Gray Level Co-occurrence Matrix (GLCM)



**UNIVERSITY
OF TURKU**

Feature extraction

- First order texture measures: statistics
 - Calculate the mean for each RGB color channel
 - Calculate the variance for each RGB color channel
- Second-order texture measures: spatial relationships between neighboring pixels
 - Calculate the correlation from the Gray-Level-CoOccurrence matrix in horizontal and vertical directions with two different reference pixel distances



Second-Order Texture Features (GLCM)

- GLCM describes texture by counting how often two neighboring pixels have certain gray-level values.
- The **correlation feature** captures how strongly pixel values are related in space.
- Horizontal and vertical directions with distances of 1 and 2 pixels are used to capture both fine and coarse textures.



Why These GLCM Distances?

- $d = 1$: captures fine local texture
- $d = 2$: captures slightly coarser texture
- Both distances are standard and interpretable



**UNIVERSITY
OF TURKU**

Template Matching

- One representative leaf image is selected as a template for each class.
- Normalized cross-correlation is used to measure how closely each image matches that template.
- The maximum response value is used as a feature, representing how similar the leaf shape is to the class prototype.

```
#Template matching
template_scores = []
for t_name, t_img in templates.items():
    t_resized = cv2.resize(t_img, (gray.shape[1], gray.shape[0]))
    res = cv2.matchTemplate(gray, t_resized, cv2.TM_CCOEFF_NORMED)
    template_scores.append(res.max())
```



**UNIVERSITY
OF TURKU**

Feature Vector

- 6 color features
- 4 GLCM correlation features
- 3 template matching scores
- Total: 13 features per image



**UNIVERSITY
OF TURKU**

Preprocessing

- Feature matrix X and label vector y
- Standardization using z-score normalization
- Ensures fair comparison between features



**UNIVERSITY
OF TURKU**

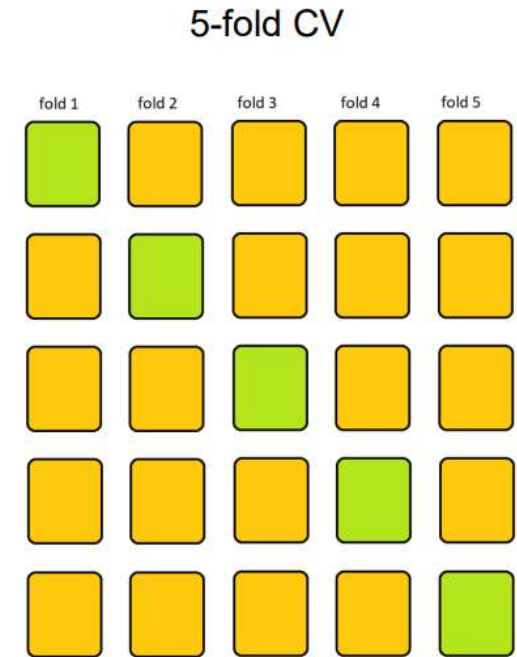
Classification Models

- Train different classifiers to identify the classes of the images
 - Ridge Classifier
 - Random Forest RF
 - Multi-Layer Perceptron MLP
- Find the best hyper-parameters for each classifier with k-fold cross validation
- Estimate the performance of the classifiers with nested cross-validation



K-fold cross validation

- A single split to test/training sets may lead to unstable results
- A procedure to estimate a model performance with unseen data (data which has not taken part in the training of the model)
- Data is split into subsets called folds
- Each fold has approximately the same size
- Each fold is used once as test data while the remaining folds are used for model training
- At each round, a performance estimation is calculated
- The total performance is calculated as the mean performance of all rounds



Model Selection

- Hyperparameter tuning is performed using GridSearch with cross-validation.
- Stratified folds are used to preserve class balance.
- Accuracy used as evaluation metric



Nested Cross-Validation

- Used to obtain an **unbiased estimate** of model performance
- Consists of **two cross-validation loops**

Inner loop (model selection):

- Uses k-fold cross-validation
- Performs **GridSearchCV**
- Selects the best hyperparameters

Outer loop (performance estimation):

- Uses a separate k-fold split
- Evaluates the model selected in the inner loop
- Estimates performance on **unseen data**
- Final performance is reported as the **mean accuracy of the outer folds**



**UNIVERSITY
OF TURKU**

Evaluation

- Check classification accuracy for each model.
- Inspect confusion matrices.
- Review classification reports.
- Analyze feature importance for Random Forest.
- Visualize the feature space using PCA.



**UNIVERSITY
OF TURKU**

About the report

- You can use the template provided in Moodle
- Write a clear report, understandable for an unspecialized reader
- Write easily readable code with comments
- If you exploit some code from web, provide a reference
- Avoid redundant code! Exploit the relevant parts and modify the code for your purposes to produce only what you need
- Everyone should submit their own implementation. It is ok to discuss the assignment with a friend. But it is not ok to copy someone's work.
- Return your output as a working Jupyter notebook
- Name your file as MLPR26_exercise_surname_firstname.jpynb



**UNIVERSITY
OF TURKU**

Grading

- The exercise covers a part of the grading in this course
- The course grade consists of the exercise and the exam, with a total score of 36 points
- Exam has 5 questions, 6 points for each
- Exercise gives 6 exam points
- From the exercise template, you can see how many exercise points can be acquired from each task.
- Exam points are given according to the table below:
 - 9–10 exercise points: 1 exam point
 - 11–12 exercise points: 2 exam points
 - 13–14 exercise points: 3 exam points
 - 15–16 exercise points: 4 exam points
 - 17–18 exercise points: 5 exam points
 - 19–20 exercise points: 6 exam points
- At least 9 exercise points are needed to pass the exercise
- The points should be distributed somewhat evenly into tasks



**UNIVERSITY
OF TURKU**

Deadline 15th of March at 16:00

- No extension granted, unless you have an extremely justified reason. In such case, ask for extension well in advance!
- Please note: Project work must be submitted before taking the e-exam.
- Start early, do not leave it to the last minute. This exercise will need some labor!
- If you encounter problems, Google first, and if you can't find an answer, ask for help
 - In the discussion area at Moodle
 - Email: svwelg@utu.fi



**UNIVERSITY
OF TURKU**